

Exploratory Data Analysis (EDA) & Data Preprocessing

Rezki Trianto
Data Science & Analytics
rezkitrianto@gmail.com



Kampus
Merdeka
INDONESIA DATA

PRAKTIKI
MENGAJAR

Hi, There!



Rezki Trianto

Linkedin : [Rezki Trianto](#)

Data Science & Analytics

Currently starting to craft amazing social impacts with



Previously with



Mentorship Program



Or just DM me

ADPList

DISCLAIMER!

Do familiar with:

- Data processing with python & pandas
- Statistics

Before this session

Outlines_

• Data Visualization Introduction - 15'

EDA & Data Preprocessing - 100'

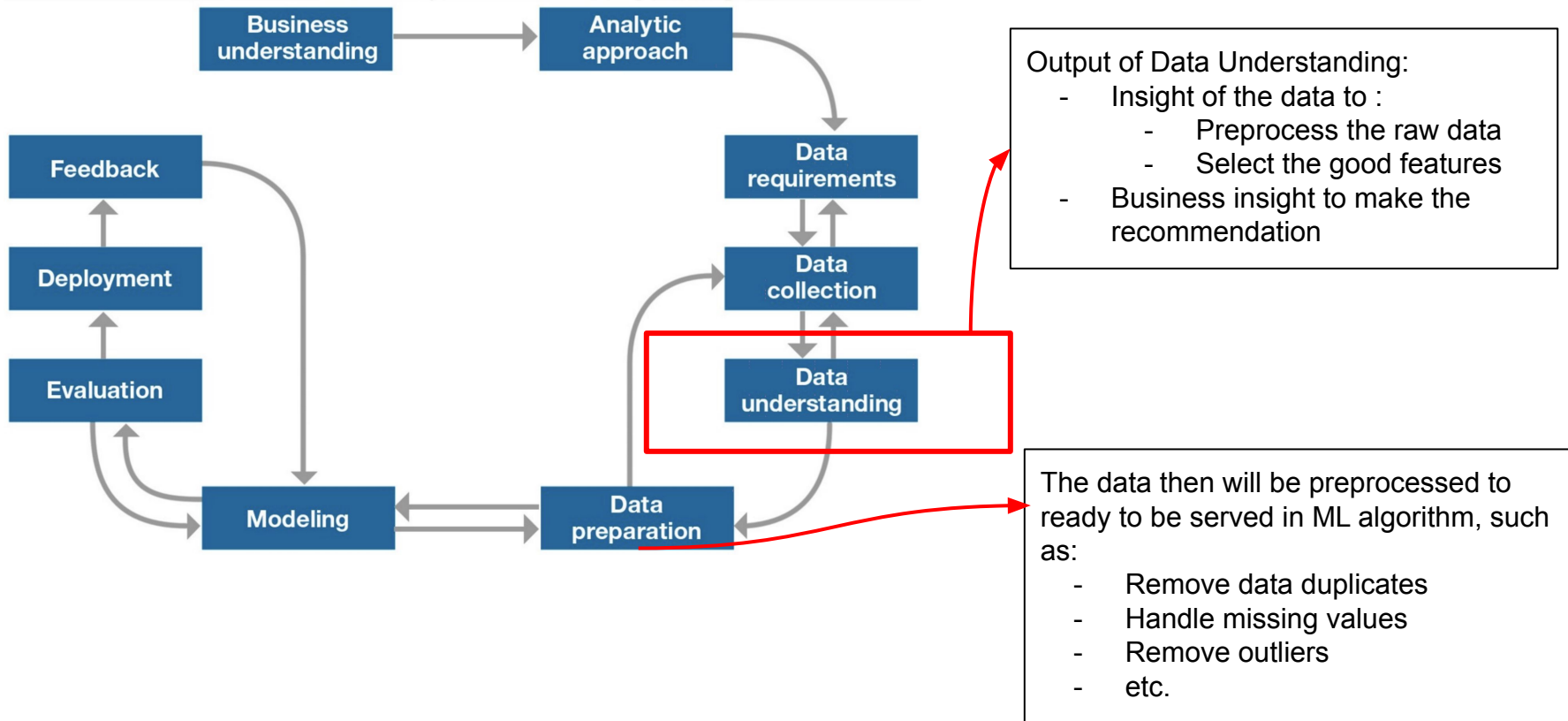
Feature Transformation 15'

Class Imbalance - 10'

QnA - 10'

•

Exploratory Data Analysis



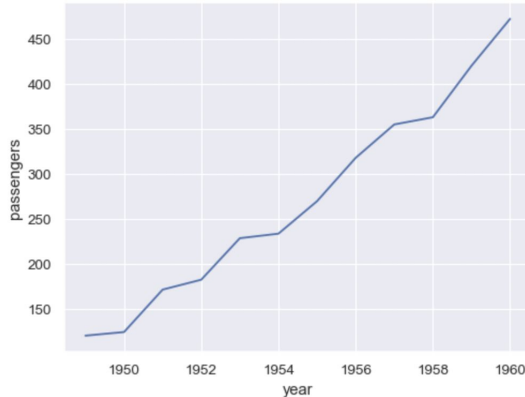
Data Visualization - Quick Intro

Library:

- **Matplotlib**: paling basic, sangat flexible
- **Seaborn**: cukup simple dan high-level; banyak digunakan untuk plot dasar statistik

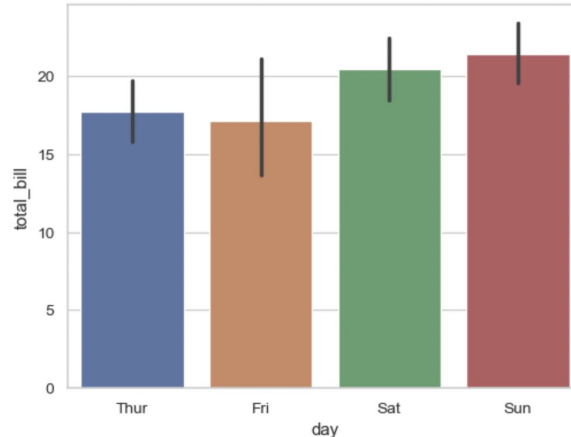
Import seaborn as sns

```
may_flights = flights.query("month == 'May'")
sns.lineplot(data=may_flights, x="year", y="passengers")
```



Line plot

```
sns.barplot(x="day", y="total_bill", data=tips)
```

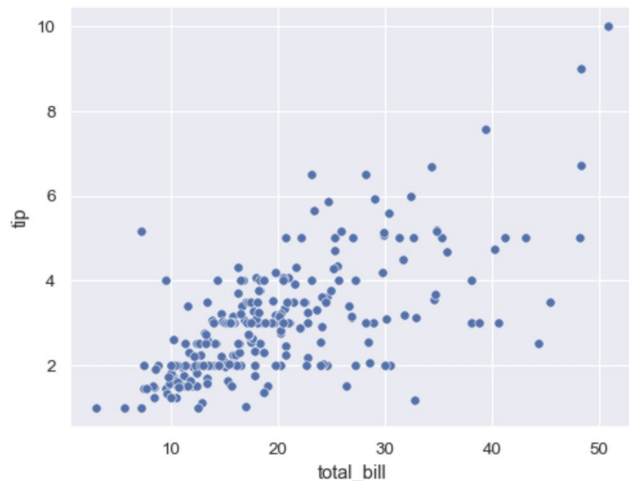


Bar plot

Data Visualization - Quick Intro

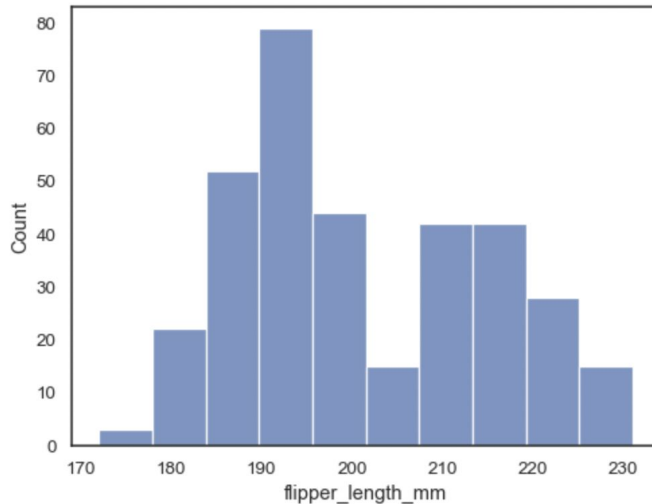
Import seaborn as sns

```
sns.scatterplot(data=tips, x="total_bill", y="tip")
```



Scatter plot

```
sns.histplot(data=penguins, x="flipper_length_mm")
```

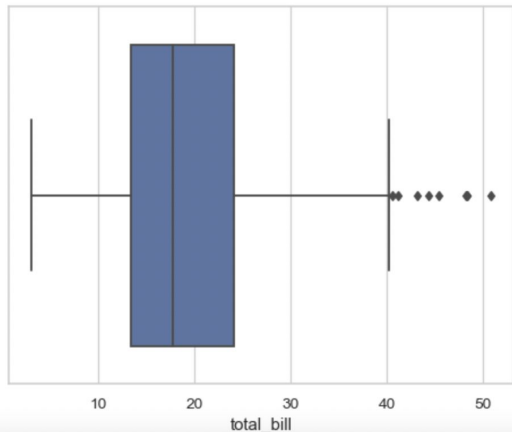


Histogram plot

Data Visualization - Quick Intro

Import seaborn as sns

```
sns.boxplot(x=tips["total_bill"])
```



Box plot

For more reference, visit seaborn documentation:

<https://seaborn.pydata.org/index.html>

Dos & Don'ts

Dos

- Check your data & data types
- Do the data still raw? Do we need to preprocess further?
- What is 1 row represent at?
- Check and remove a duplicate data
- What available features that we have?
- Check overall data distribution & choose a good features
- Based on data condition analysis, plan the preprocessing part

Dont's

- Do preprocessing before completing the EDA part
- Too focus to looking at business insight

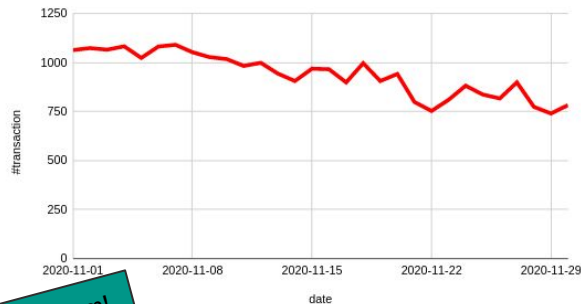
How to EDA's?

1. Descriptive Statistics
2. Univariate Analysis
3. Multivariate Analysis

Remember our Lovely Case Study last week? ❤️

Descriptive Analytics (What happened?)

Daily #Transaction



Problem!

“Whoops! Our daily transaction declining over the weeks!”

Diagnostic Analytics (How come this happened?)

- **Promo performance declining from push notification channel. The targeted user is unmatched**

Predictive Analytics (Predict what will be happened?)

- Develop a propensity model to detect customer who really interested to respond the campaign until make the transaction
(classification)

Let's use this sample to do our EDA

Use Case: Propensity Models

How we predict whether the user will make a transaction by looking their previous behavior?

Dataset: [Kaggle](#)

Customer propensity to purchase dataset

A data set logging shoppers interactions on an online store



[Data](#) [Code \(6\)](#) [Discussion \(2\)](#) [Metadata](#)

About Dataset

Context

You get many visitors to your website every day, but you know only a small percentage of them are likely to buy from you, while most will perhaps not even return. Right now you may be spending money to re-market to everyone, but perhaps we could use machine learning to identify the most valuable prospects?

Content

This data set represents a day's worth of visit to a fictional website. Each row represents a unique customer, identified by their unique `UserID`. The columns represent feature of the users visit (such as the device they were using) and things the user did on the website in that day. These features will be different for every website, but in this data a few of the features we consider are:

Usability ⓘ

10.00

License

[CC0: Public Domain](#)

Expected update frequency

Never

1. EDA: Descriptive Statistics

a. Understand your data better

About this file				
This testing sample contains less rows that the training set and can be used to validate your model.				
A UserID	# basket_icon_click	# basket_add_list	# basket_add_detail	# sort_by
A unique identifier for the visitor	Did the visitor click on the shopping basket icon?	Did the visitor add a product to their shopping cart on the 'list' page?	Did the visitor add a product to their shopping cart on the 'detail' page?	Did the visitor sort products on a page?
151655 unique values				
9d24-25k4-47889d24-25k4-494b-398124	0	0	0	0
7732-1k58-47887732-1k58-4475-679678	0	0	0	0
94k2-632j-471394k2-632j-4b4j-228160	0	0	0	0
jdd8-419d-4714jdd8-419d-4198-674376	0	0	1	0

Let's refer to our [Kaggle](#) dataset
Do understand each definition of your feature just like you understand your partner

- Know the definition
- Know the data types
- Know the values of each features
- Know the number of rows

1. EDA: Descriptive Statistics

b. Understand your data better - df.info()

```
1 df.info()

executed in 125ms, finished 11:32:30 2022-08-21

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 455401 entries, 0 to 455400
Data columns (total 25 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   UserID                455401 non-null object
 1   basket_icon_click     455401 non-null int64
 2   basket_add_list       455401 non-null int64
 3   basket_add_detail     455401 non-null int64
 4   sort_by               455401 non-null int64
 5   image_picker          455401 non-null int64
 6   account_page_click    455401 non-null int64
 7   promo_banner_click    455401 non-null int64
 8   detail_wishlist_add   455401 non-null int64
 9   list_size_dropdown    455401 non-null int64
10   closed_minibasket_click 455401 non-null int64
11   checked_delivery_detail 455401 non-null int64
12   checked_returns_detail 455401 non-null int64
13   sign_in               455401 non-null int64
14   saw_checkout          455401 non-null int64
15   saw_sizecharts        455401 non-null int64
16   saw_delivery          455401 non-null int64
17   saw_account_upgrade   455401 non-null int64
18   saw_homepage          455401 non-null int64
19   device_mobile         455401 non-null int64
20   device_computer       455401 non-null int64
21   device_tablet         455401 non-null int64
22   returning_user        455401 non-null int64
23   loc_uk                455401 non-null int64
24   ordered               455401 non-null int64

dtypes: int64(24), object(1)
memory usage: 86.9+ MB
```

- Check whether we have null values here?
- Check whether we have incomplete data?
- Check inconsistent data type
 - (i.e. basket_icon_click will be wrong if have string data type)

What if we have NULL/Incomplete data?

- Drop if the number is too large (more than 40%)

```
1 df.drop(['basket_icon_click'], axis=1)
```

- Or if the number is not too large, drop only the rows

```
1 df.dropna(subset=['basket_icon_click', 'basket_add_list'])
```

- Change to mean if the distribution is not skewed (for numeric types)
- Change to median if the distribution is quite skewed (for numeric types)
- Change to mode for categorical data
- Build prediction by using ML, use it's prediction result (regression for numeric, classification for categorical)

```
1 df = df.fillna(df.mean())  
2 df = df.fillna(df.min())  
3 df = df.fillna(df.max())  
4 df = df.fillna(0)
```

What if we have inconsistent data type?

- Change the data type as you wish

```
1 df['basket_icon_click'] = df['basket_icon_click'].astype('str')
```


1. EDA: Descriptive Statistics

c. Understand your data better - `df.sample()`

- Understand the values of your data. Is it make sense?
- Do this repeated times until we sure that our data is valid and safe

```
1 df.sample(5)
```

executed in 65ms, finished 19:18:23 2022-08-26

	UserID 🚩	basket_icon_click 🚩	basket_add_list 🚩	basket_add_detail 🚩	sort_by 🚩	image_picker 🚩	account_page_click 🚩	promo_banner_click 🚩
9641-1e5d8305-9641-467d-9672-423886	0	0	0	1	1	0	0	
8646-5d870859-8646-4633-9136-559321	0	0	0	0	0	0	0	
7ba8-c48cc51e-7ba8-4c98-81b7-214550	0	0	0	0	0	0	0	
9019-31e58a03-9019-40db-a470-123878	0	0	0	0	0	0	0	
4765-76ab0367-4765-4d63-b735-909439	1	0	1	0	0	0	0	

1. EDA: Descriptive Statistics

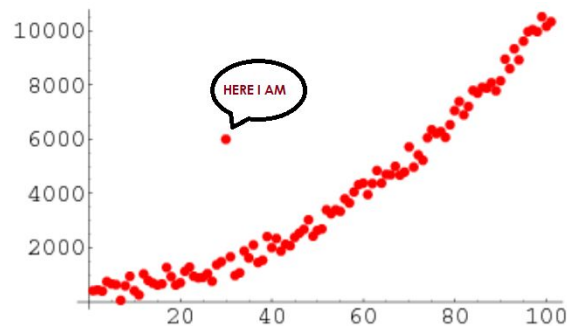
d. Understand your data better - `df.describe()`

- Understand distribution of our data
- Do we have outlier? (Check between mean and median)
- Do the value make sense? (Check min and max)

```
1 df.describe().transpose()
```

executed in 189ms, finished 19:20:20 2022-08-26

	count	mean	std	min	25%	50%	75%	max
basket_icon_click	455401.0	0.099150	0.298864	0.0	0.0	0.0	0.0	1.0
basket_add_list	455401.0	0.074521	0.262617	0.0	0.0	0.0	0.0	1.0
basket_add_detail	455401.0	0.112916	0.316490	0.0	0.0	0.0	0.0	1.0
sort_by	455401.0	0.036849	0.188391	0.0	0.0	0.0	0.0	1.0
image_picker	455401.0	0.026735	0.161307	0.0	0.0	0.0	0.0	1.0
account_page_click	455401.0	0.003570	0.059647	0.0	0.0	0.0	0.0	1.0
promo_banner_click	455401.0	0.016208	0.126274	0.0	0.0	0.0	0.0	1.0
detail_wishlist_add	455401.0	0.003511	0.059151	0.0	0.0	0.0	0.0	1.0
list_size_dropdown	455401.0	0.230362	0.421065	0.0	0.0	0.0	0.0	1.0
closed_minibasket_click	455401.0	0.017277	0.130302	0.0	0.0	0.0	0.0	1.0



Outlier example

- Since our data is mostly categorical,
 - cast this first into categorical, and do the `.describe()` again
 - Do a feature transformation of categorical data to numerical*

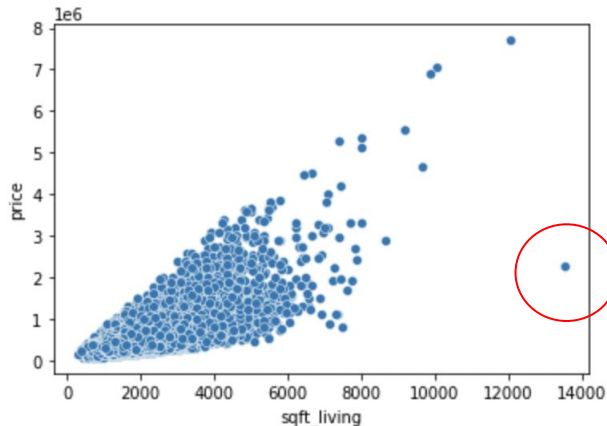
1. EDA: Descriptive Statistics

To check an outlier, use scatterplot or boxplot. Example:

```
1 sns.scatterplot(data=dataset, x='sqft_living', y='price')
```

executed in 131ms, finished 18:53:07 2022-09-04

<AxesSubplot:xlabel='sqft_living', ylabel='price'>

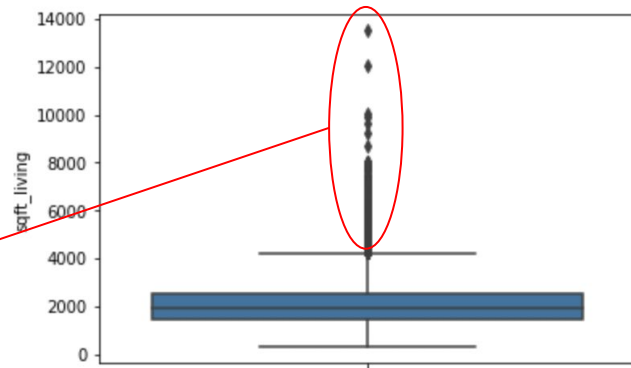


These thing might
be an outlier

```
1 sns.boxplot(data=dataset, y='sqft_living')
```

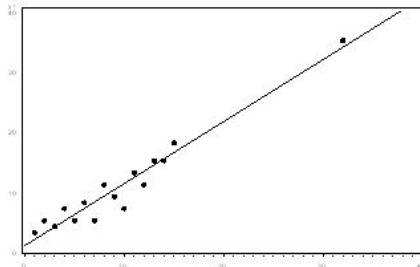
executed in 86ms, finished 18:56:35 2022-09-04

<AxesSubplot:ylabel='sqft_living'>

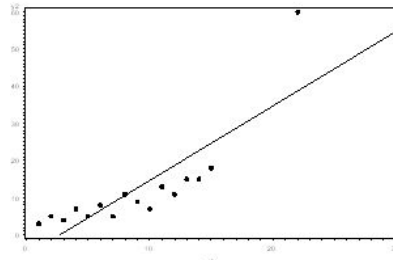


What if we have Outliers?

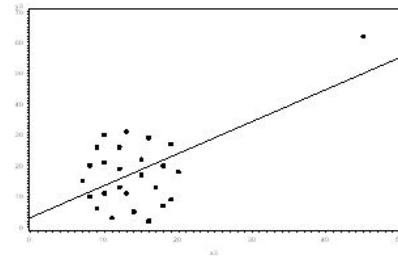
- Understand the outlier
 - If it because of the data input error, drop it (e.g. Height: -10 cm)
 - If it's a special and unique case, think to keep it
- For a machine learning modeling purpose, only drop outliers in data training
- (a) If the outliers not affect the result, but violates the assumptions, drop it
- (b) If the outliers affect the result, but violates the assumptions, drop it
- (c) If the outliers affect the correlation result, drop it; don't summarize any correlation in our data



(a) good result;
violates assumption



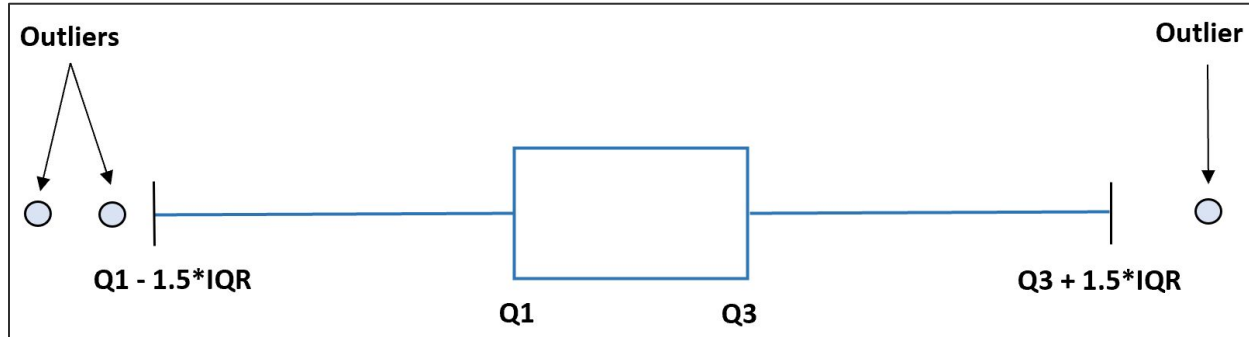
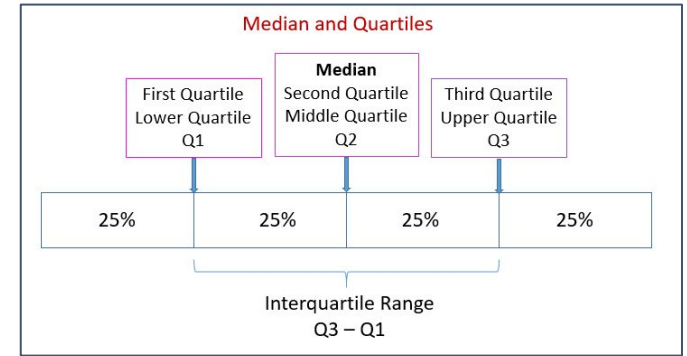
(b) bad result;
violates assumption



(c) outliers affect the
correlation

How to drop outliers?

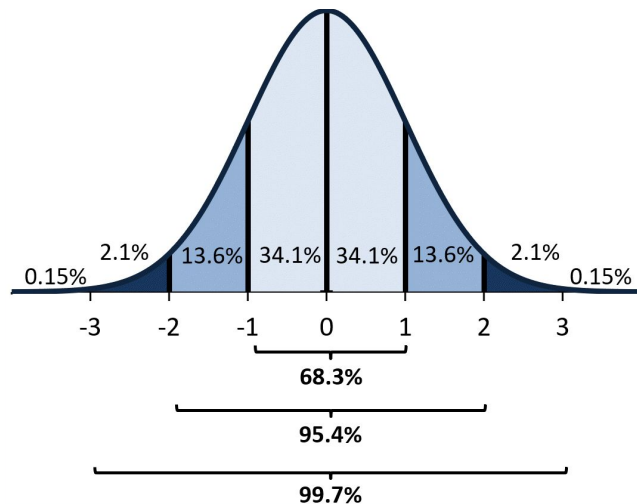
- IQR (interquartile) Method
 - $IQR = Q3 - Q1$



```
1 Q1 = df['umur'].quantile(0.25)
2 Q3 = df['umur'].quantile(0.75)
3 IQR = Q3 - Q1
4 low_limit = Q1 - (1.5 * IQR)
5 high_limit = Q3 + (1.5 * IQR)
6 filtered_entries = ((df['umur'] >= low_limit) & (df['umur'] <= high_limit))
7 df = df[filtered_entries]
```

How to drop outliers?

- Z-score Method
 - Z-score: berapa kali standard deviation jarak sebuah nilai dari rata-rata kolom

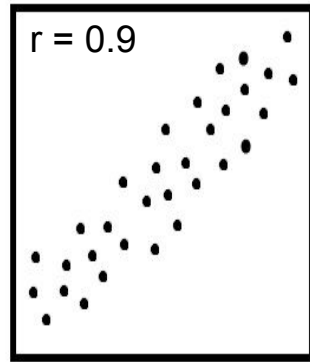


$$z = \frac{x - \mu}{\sigma}$$

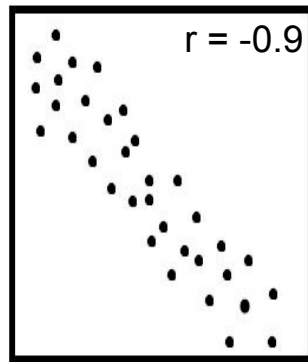
- Outlier if absolute of z-score > 3; So we remove ~0.3% of data of the most extreme. This usually used when our data is normally distributed

```
1 from scipy import stats
```

```
1 z_scores = np.abs(stats.zscore(df['umur']))  
2 filtered_entries = (z_scores < 3)  
3 df = df[filtered_entries]
```

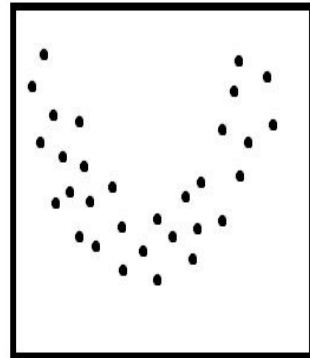


positive linear
association



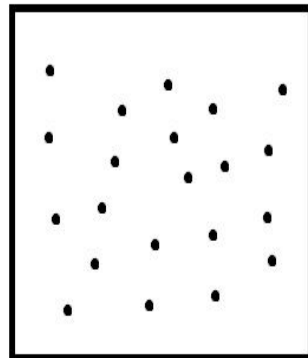
negative linear
association

$r = 0$



nonlinear
association

$r = 0$



no association

Correlation Analysis

- Talking about the correlation, higher correlation value (r) means, it have a linear association in a positive way or negative way.
- While a lower correlation means it doesn't have any association between the features

```
1 df.corr()
```

executed in 694ms, finished 06:46:38 2022-09-06

	↕ basket_icon_click ↕	basket_add_list ↕	basket_add_detail ↕
basket_icon_click	1.000000	0.466671	0.529947
basket_add_list	0.466671	1.000000	0.340968
basket_add_detail	0.529947	0.340968	1.000000
sort_by	0.073016	0.106852	0.085854

1. EDA: Descriptive Statistics

d. Understand your data better - `df.describe()`

- Check the distribution, do we have a feature that have a big number of unique value; or a feature with a very dominate value in single value

```
1 cols = df.columns.to_list()
2 df = df[cols].astype(str)
```

executed in 3.15s, finished 19:27:43 2022-08-26

```
1 df.describe().transpose()
```

executed in 1.40s, finished 19:27:46 2022-08-26

	count	unique	top	freq
UserID	455401	455401	a720-6b732349-a720-4862-bd21-644732	1
basket_icon_click	455401	2	0	410248
basket_add_list	455401	2	0	421464
basket_add_detail	455401	2	0	403979
sort_by	455401	2	0	438620
image_picker	455401	2	0	443226
account_page_click	455401	2	0	453775
promo_banner_click	455401	2	0	448020

- Check per features

```
1 df['ordered'].value_counts()
```

executed in 69ms, finished 19:33:24 2022-08-26

0 436308

1 19093

Name: ordered, dtype: int64

*Imbalance data example***

1. EDA: Descriptive Statistics

e. Do we have a duplicate data? - `df.duplicated()`

```
1 df.duplicated().sum()
```

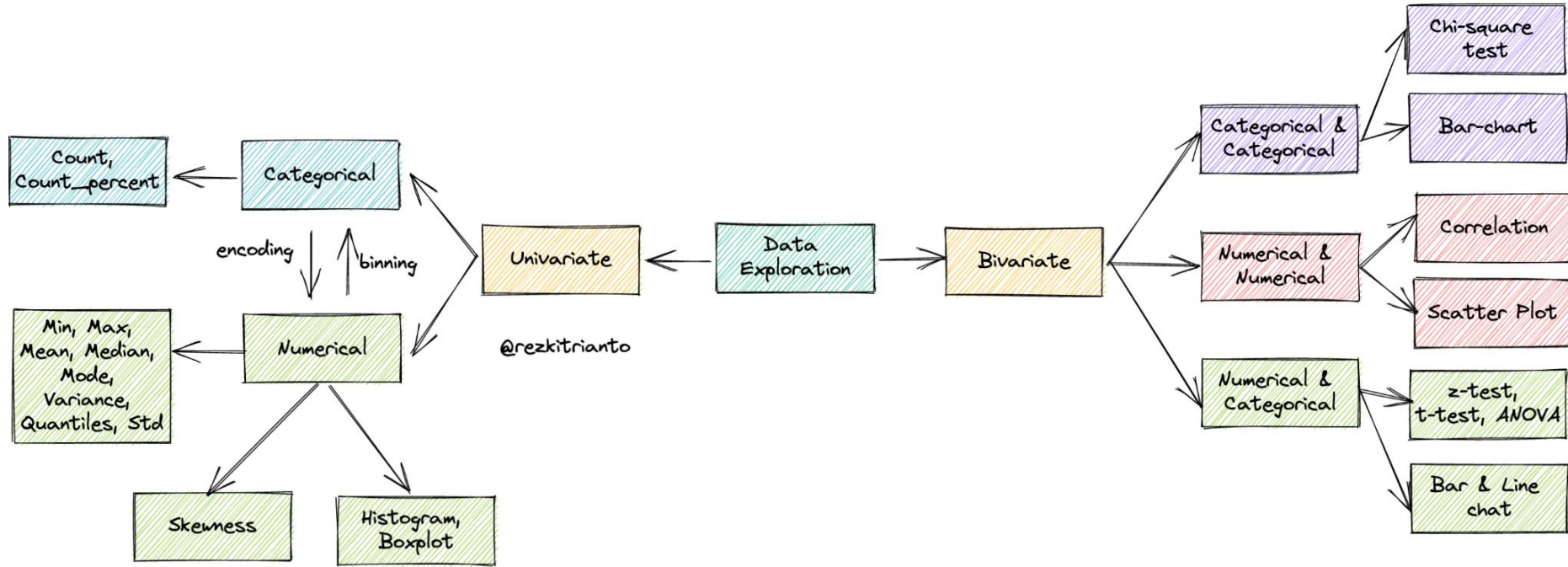
executed in 326ms, finished 07:02:11 2022-09-06

If we have a duplicate data, remove it.

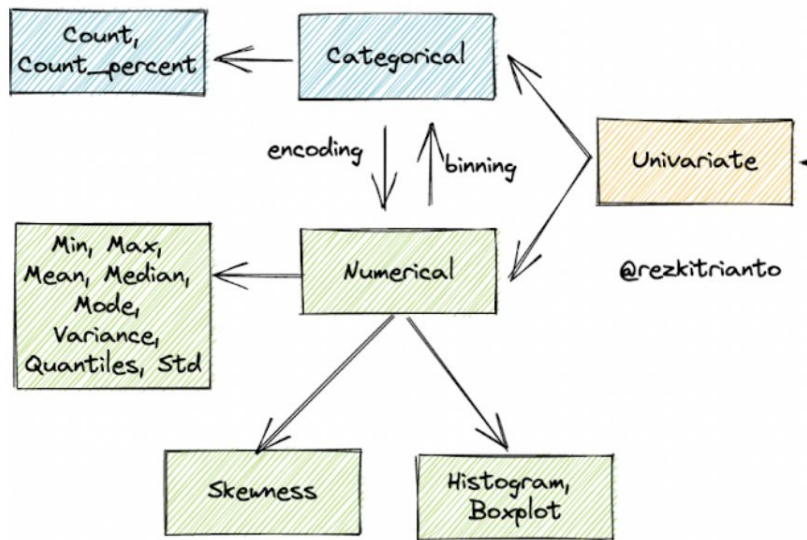
```
1 df = df.drop_duplicates()
```

executed in 381ms, finished 07:03:33 2022-09-06

Data Exploration



2. EDA: Univariate Analysis



2. EDA: Univariate Analysis

- a. Check distribution
 - i. For numeric: Check distribution, histogram/boxplot
 - 1. Do we have outliers?
 - 2. Do we have a skewed distribution?
 - ii. For categorical: Check count. Do we have a major value from a single feature?

```
1 df.describe().transpose()
```

executed in 189ms, finished 19:20:20 2022-08-26

	count	mean	std	min	25%	50%	75%	max
basket_icon_click	455401.0	0.099150	0.298864	0.0	0.0	0.0	0.0	1.0
basket_add_list	455401.0	0.074521	0.262617	0.0	0.0	0.0	0.0	1.0
basket_add_detail	455401.0	0.112916	0.316490	0.0	0.0	0.0	0.0	1.0
sort_by	455401.0	0.036849	0.188391	0.0	0.0	0.0	0.0	1.0
image_picker	455401.0	0.026735	0.161307	0.0	0.0	0.0	0.0	1.0
account_page_click	455401.0	0.003570	0.059647	0.0	0.0	0.0	0.0	1.0
promo_banner_click	455401.0	0.016208	0.126274	0.0	0.0	0.0	0.0	1.0
detail_wishlist_add	455401.0	0.003511	0.059151	0.0	0.0	0.0	0.0	1.0
list_size_dropdown	455401.0	0.230362	0.421065	0.0	0.0	0.0	0.0	1.0
closed_minibasket_click	455401.0	0.017277	0.130302	0.0	0.0	0.0	0.0	1.0

```
1 df['ordered'].value_counts()
```

executed in 69ms, finished 19:33:24 2022-08-26

```
0    436308
1     19093
Name: ordered, dtype: int64
```

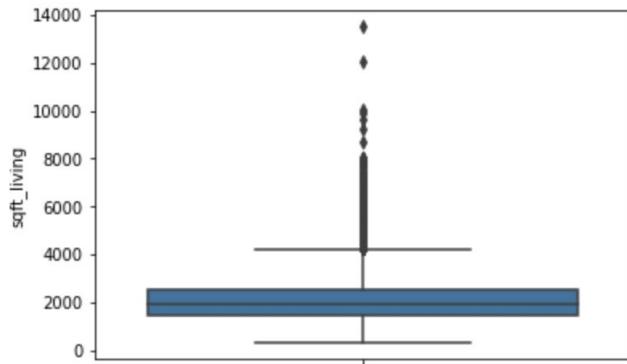
2. EDA: Univariate Analysis

Example of histogram & Boxplot

```
1 sns.boxplot(data=dataset, y='sqft_living')
```

executed in 86ms, finished 18:56:35 2022-09-04

<AxesSubplot:ylabel='sqft_living'>

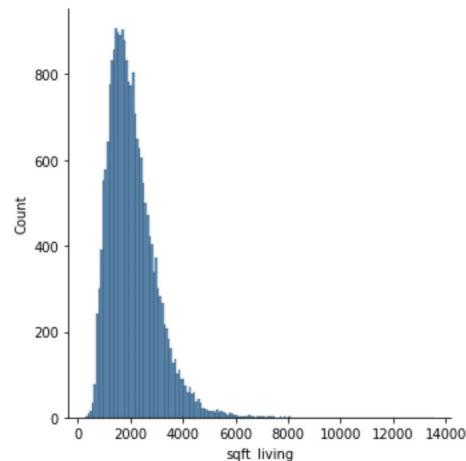


Have outliers. Think to remove it

```
1 sns.displot(data=dataset, x='sqft_living')
```

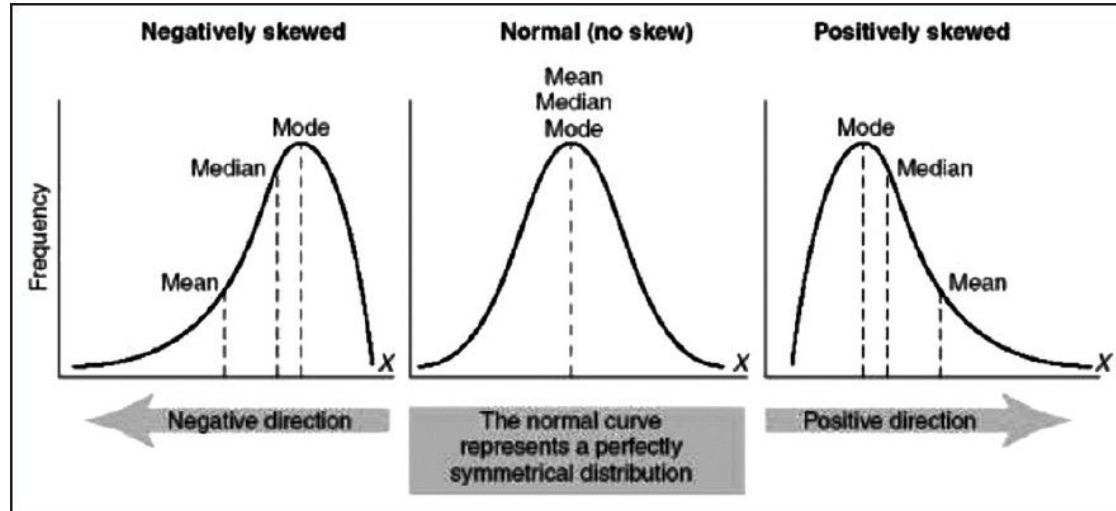
executed in 278ms, finished 18:58:42 2022-09-04

<seaborn.axisgrid.FacetGrid at 0x7f949af2b8e0>



Right skewed distribution. Think to normalize it

Distribution Types



- A negative or positive skewed distribution usually affected by an outliers

What if we have a skewed distribution?

- It's better to make it normal
 - Normal distribution helps us get rid of the outliers and makes inferential calculations much easier as it is faster to compare two normally distributed variables than two following different distributions which is very common in real world analysis.
- So we need a feature transformation between:
 - Normalization
 - Standardization
 - Log Transform
- We need to scale & applied these transformation to:
 - Make sure all feature will be treated uniformly
 - Faster to machine learning model to convergence

Feature Transformation: Numerical

	Normalization	Scaling	Log Transform
Definition	Change feature scale to rigid range (usually 0-1)	Change feature scale with no rigid range (mean = 0; std = 1)	Apply a logarithm function to transform a value
Effect	Not changing the distribution shape	Change to (near) normal distribution	- Change to (near) normal distribution - Usually used in a right skewed distribution
Implementation	<pre>from sklearn.preprocessing import MinMaxScaler m = MinMaxScaler() scaler = m.fit(X_train[['sqft_living']]) X_train['sqft_living'] = scaler.transform(X_train[['sqft_living']])</pre>	<pre>from sklearn.preprocessing import StandardScaler m = StandardScaler() scaler = m.fit(X_train[['sqft_living']]) X_train['sqft_living'] = scaler.transform(X_train[['sqft_living']])</pre>	<pre>1 import numpy as np 2 X_train['sqft_living'] = np.log(X_train['sqft_living'])</pre>

Feature Transformation: Categorical*

Change categorical to numerical -> so all ML algorithm can process & calculate this

- Label Encoding (usually used when we have ordinal values)

```
mapping_pendidikan = {  
    'SMA': 0,  
    'S1': 1,  
    'S2': 2,  
    'S3': 3  
}
```



pendidikan
1
0
1
2
3

```
df['pendidikan'] = df['pendidikan'].map(mapping_pendidikan)
```

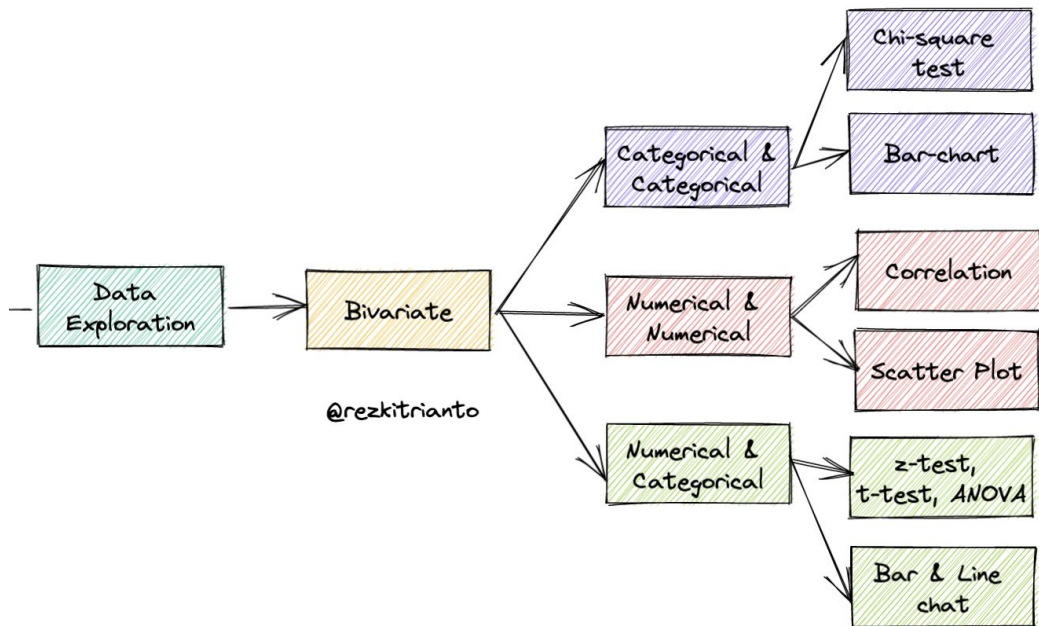
- One Hot Encoding (Change feature categorical into each single column with binary values)

```
1 pd.get_dummies(df['pekerjaan'], prefix='kerja')
```



	kerja_FREELANCE	kerja_PNS	kerja_SWASTA
0	0	0	1
1	0	1	0
2	0	1	0
3	1	0	0
4	0	1	0
5	0	0	1
6	1	0	0

3. EDA: Bivariate Analysis



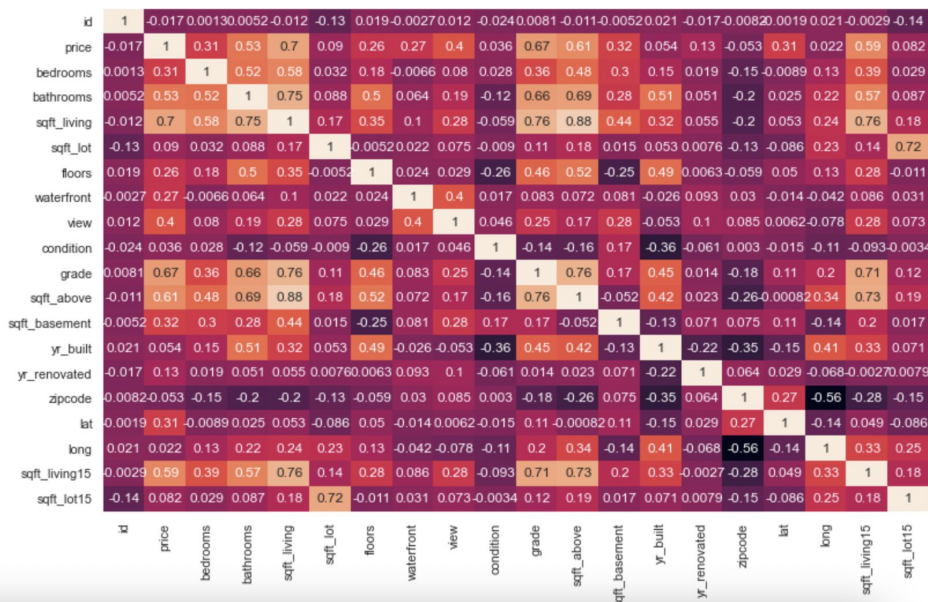
3.1. EDA: Bivariate Analysis

Numerical vs. Numerical: Correlation Analysis

```
2 sns.heatmap(dataset.corr(), annot=True)
```

executed in 1.03s, finished 13:36:45 2022-02-06

<AxesSubplot:>



What to check (esp. for feature selection)

- Do the feature have a correlation with a target (correlation >0.5 is marked as a high correlation)
- Do we have 2 features with a strong correlation value (>0.7); If so, think to drop it

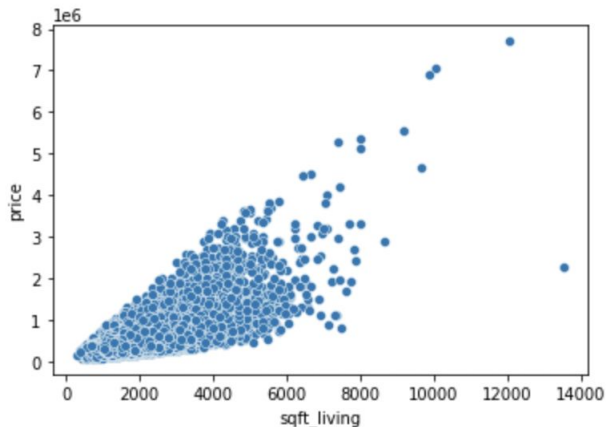
3.1. EDA: Bivariate Analysis

Numerical vs. Numerical: Scatter Plot

```
1 sns.scatterplot(data=dataset, x='sqft_living', y='price')
```

executed in 131ms, finished 18:53:07 2022-09-04

<AxesSubplot:xlabel='sqft_living', ylabel='price'>



What to check:

- Same with the correlation plot, a higher correlation will draw a linear relationship between features (higher x will contribute to higher y)

3.2. EDA: Bivariate Analysis

Categorical vs. Categorical

- a. Do a chi-square test ([Reference](#))
 - i. Do a label encoder transformation before do a chi-square test

```
1 from sklearn.feature_selection import chi2
2
3 X = df.drop(columns=['ordered', 'UserID'], axis=1)
4 y = df['ordered']
5 chi2(X, y)
```

executed in 141ms, finished 08:35:01 2022-09-06

```
(array([7.52682557e+04, 3.48768184e+04, 6.93808424e+04, 1.30931917e+03,
        2.26537987e+03, 1.48878713e+03, 1.43184576e+03, 2.50964113e+02,
        8.40617614e+03, 8.77307552e+03, 2.72258524e+05, 1.59656129e+03,
        1.83811194e+05, 2.10579577e+05, 2.59353867e+01, 4.48252173e+02,
        3.04151149e+02, 8.04875863e+03, 2.67689696e+02, 8.88563693e+02,
        1.13898313e+02, 7.69984766e+02, 3.04483250e+01]),
 array([0.00000000e+000, 0.00000000e+000, 0.00000000e+000, 1.06669420e-286,
        0.00000000e+000, 0.00000000e+000, 0.00000000e+000, 1.60050059e-056,
        0.00000000e+000, 0.00000000e+000, 0.00000000e+000, 0.00000000e+000,
        0.00000000e+000, 0.00000000e+000, 3.53037795e-007, 1.73178032e-099,
        4.10578356e-068, 0.00000000e+000, 3.61774346e-060, 3.00575781e-195,
        1.37165290e-026, 1.81162860e-169, 3.42879262e-008]))
```

Chi square test values

Lower value means the feature
similar to each other

p-values

Pick the high p-values to drop the features; usually > 0.05 to
understand that the feature is not statistically significant to be a
predictor of the target

3.3. EDA: Bivariate Analysis

Numerical vs. Categorical

- a. Do statistical test of t-test, z-test, ANOVA ([Reference](#))

	t-test	z-test	ANOVA
Number of features	Compare 2 different categorical features with bigger sample size (more than 30)	Compare 2 different categorical features with small sample size (less than 30)	Compare >2 different categorical features
Purpose	Whether the feature is a good predictor to the target; and whether it's a statistically significant		

Feature Selection after Bivariate analysis

- Pick your features for training the machine learning model

```
1 X = df[['basket_icon_click', 'basket_add_list', 'basket_add_detail']]  
2 y = df[['ordered']]
```

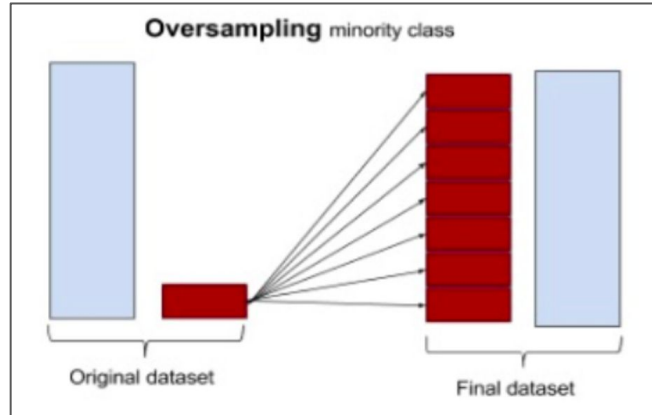
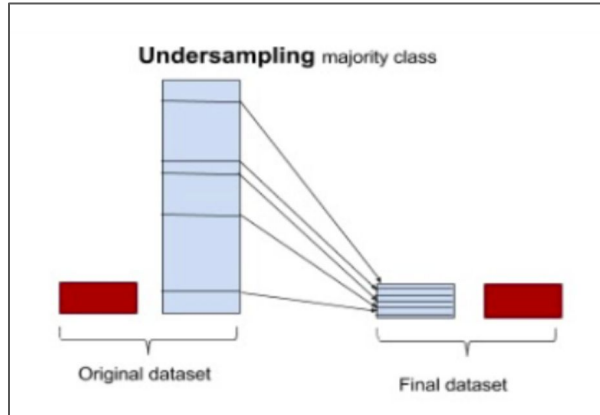
executed in 30ms, finished 06:37:51 2022-09-06

Class Imbalance**

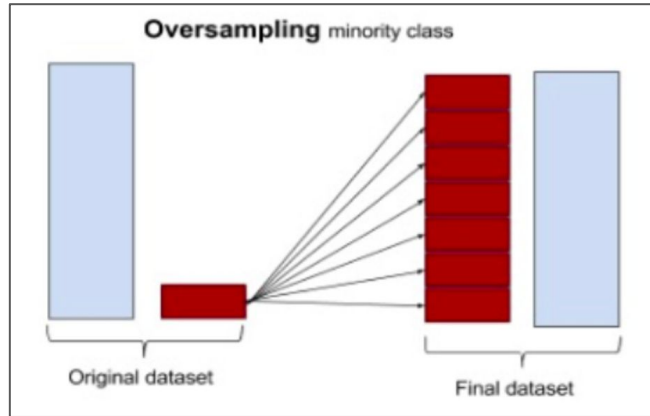
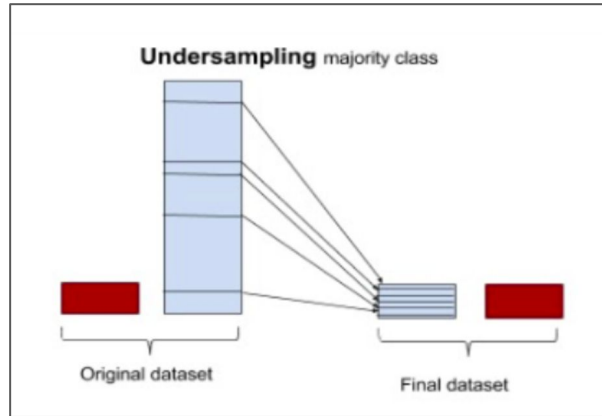
Will make the machine learning model train not fair.

How to handle this:

- Use more proper evaluation method for machine learning (F1/ROC-AUC/PR-AUC)
- Do undersampling or oversampling



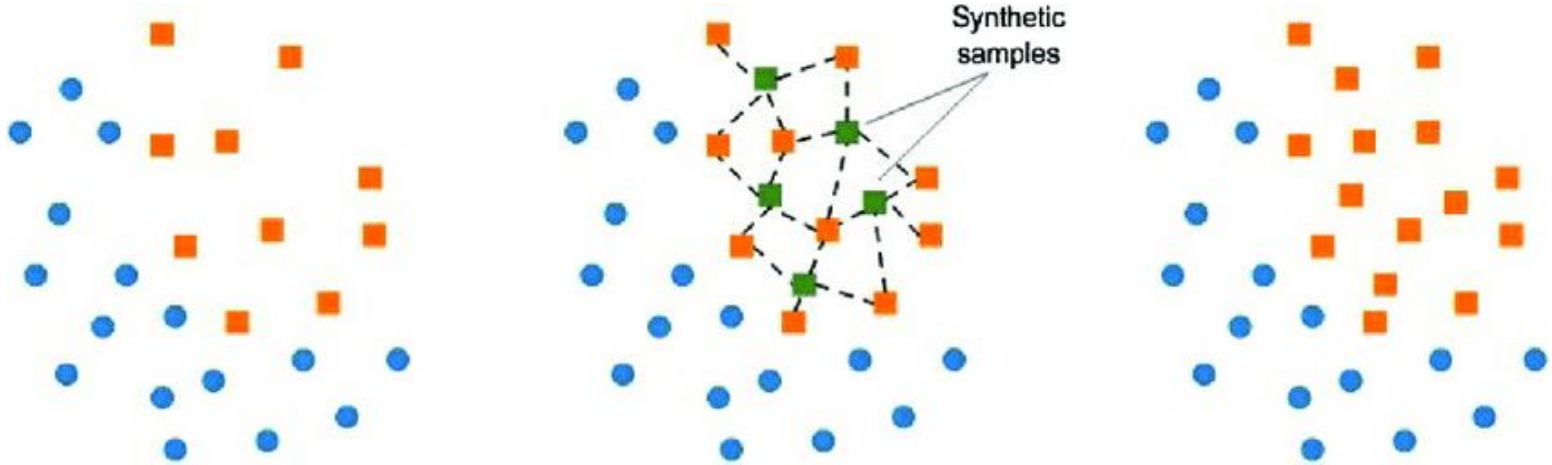
Oversampling & Undersampling



```
1 from imblearn import under_sampling, over_sampling
2 X_under, y_under = under_sampling.RandomUnderSampler(0.5).fit_resample(X, y)
3 X_over, y_over = over_sampling.RandomOverSampler(0.5).fit_resample(X, y)
4 X_over_SMOTE, y_over_SMOTE = over_sampling.SMOTE().fit_resample(X, y)
```

SMOTE

Make a synthetic data that similar with the previous one



Q&A Session